

language that will handle the logic. I am going to use Python in this article, but you can use any language that you like. So, in theory, your program adds tasks to your queue from one end, and another program takes out tasks from the other end of the queue and runs them.

Redis + HotQueue

HotQueue is a small Python library that implements queues with Redis, and is really easy to get started with. Let's directly dive into the code:

```
# Producer script (or the script that adds tasks
# in the queue)

from hotqueue import HotQueue
queue = HotQueue("myqueue", host="localhost", port=6379,
                db=0)

queue.put('some message/work')
```

The above example is very simple. There is a *producer* (a script that adds tasks in the queue) and a *consumer* (a script that picks up tasks from the other end of the queue, and does the work accordingly). In the *producer* script, we first connect with the Redis server, and then add the task (or message) using the *put* method of the *HotQueue* class. So whenever we run the producer script, a task gets added. The *consumer* script starts by connecting to the Redis server, and then goes into an infinite loop. It first tries to get a message from the queue by using the *get* method of the *HotQueue* class. This is a blocking call, which means that nothing will execute unless you have a task in the queue. As soon as a message is picked up from the queue, execution proceeds, and you may do anything with the picked up task. So in our email example, you may use the *put* method in, say, your *Flask* views. The actual task may be a JSON-encoded message with the recipient's email address, the message and the subject; and our *worker* script will pick that up and send the email.

What is Flask?

Flask is a lightweight web micro-framework written in Python. It does pretty much what any micro-framework (or framework), like Sinatra (written in Ruby) and Silex (written in PHP), does. In the context of this article, we will be using Flask to write request handlers that will add tasks to queues. In the next code snippet, *submit_order()* is a request handler for all incoming POST requests at */order/* add URI. So whenever a POST request is made to */order/* add, *submit_order()* is called.

```
# Consumer script (or the script that picks up
# tasks from the queue)
```

```
from hotqueue import HotQueue
queue = HotQueue("myqueue", host="localhost", port=6379,
                db=0)
```

```
while True:
    message = queue.get()
    # do something awesome with message
```

In your *Flask* app, connect with the Redis server using the *HotQueue* class whenever you need to queue anything, then JSON-encode the task, and queue it using the *put* method from the *HotQueue* class.

```
import json
from hotqueue import HotQueue

@app.route('/order/add', method=['POST'])
def submit_order():
    # create entries in your database

    queue = HotQueue("myqueue", host="localhost",
                    port=6379, db=0)
    message = dict(recipient=g.user['email'],
                  subject='Order confirmation.',
                  body='<Message in HTML or text>')
    queue.put(json.dumps(message))

    return render_template('thanks_order.html')
```

In your worker process, pick up a task/message from the queue, decode the JSON-encoded task into a Python native dictionary, and send the email using any Python library that works for you. Now all you have to do is fire up your worker processes on your machine, and let requests come in! Using *HotQueue* is that simple! As I said, you can implement your own queue in about 15 minutes—and if you do so, the end result will be something pretty close to *HotQueue*.

Redis + PyRes

HotQueue is simple, and it just works. *PyRes*, on the other hand, is more robust. It is a Python clone of Github's *Resque* (written in Ruby) and offers more features like displaying the status of queues, failure handling, etc. Since *PyRes* is a clone of *Resque*, it honours the naming conventions *Resque* makes use of and, therefore, it makes it possible to use *Resque*'s Web monitoring system *god* and *monit*. These monitoring systems can be used not only to check the status of your queues, but also to fire up new workers, or kill stale workers, as and when necessary. Here is a sample code of our email task example:

```
# Consumer script (or the script that picks up
# tasks from the queue)
```